Mình là thực tập sinh, chưa từng làm TDD. Giải thích Test-Driven Development là gì, Red
– Green – Refactor nghĩa là gì. Dùng ví dụ đơn giản, không dùng framework phức tạp.

TDD là gì?

Test-Driven Development (phát triển hướng kiểm thử) là cách viết code mà **bạn viết test trước, rồi mới viết code**. Thông thường ta viết code xong mới kiểm tra. TDD làm ngược lại: trước tiên mô tả "code phải làm được gì" bằng một bài test, sau đó mới viết code để đáp ứng đúng điều đó.

Lợi ích chính:

- Bạn phải nghĩ rõ yêu cầu trước khi code.
- Code luôn có test đi kèm, nên sửa sau này ít sợ làm hỏng chỗ khác.
- Mỗi lần chỉ làm một bước nhỏ, dễ kiểm soát.

Red – Green – Refactor

Đây là vòng lặp ngắn bạn lặp đi lặp lại:

1. **Red (Đỏ)**: Viết một test cho tính năng *chưa có*. Chạy test, nó **phải fail**. Fail chứng tỏ test thật sự kiểm tra được điều gì đó.
2. **Green (Xanh)**: Viết code **ít nhất có thể** để test pass. Chưa cần đẹp, chỉ cần đúng.
3. **Refactor (Dọn dẹp)**: Code đã chạy đúng, giờ làm nó gọn gàng, dễ đọc hơn. Chạy lại test để chắc vẫn xanh.

Rồi quay lại bước 1 với yêu cầu tiếp theo.

Ví dụ: hàm FizzBuzz (Python thuần, chỉ dùng `assert`)

Yêu cầu: hàm `fizzbuzz(n)` trả về `"Fizz"` nếu n chia hết cho 3, `"Buzz"` nếu chia hết cho 5, `"FizzBuzz"` nếu chia hết cho cả hai, còn lại trả về số dưới dạng chuỗi.

Vòng 1

Red — viết test trước, khi hàm còn chưa tồn tại:

```python
def test_so_thuong():
    assert fizzbuzz(1) == "1"

test_so_thuong()
```

Chạy: lỗi `NameError: fizzbuzz is not defined`. Đỏ. ✅ (đúng như mong đợi)

Green — viết code tối thiểu:

```python
def fizzbuzz(n):
    return str(n)
```

Chạy lại: pass. Xanh.

Refactor — code còn quá ngắn, chưa có gì để dọn. Bỏ qua.

Vòng 2

Red:

```python
def test_chia_het_cho_3():
    assert fizzbuzz(3) == "Fizz"
```

Chạy: fail, vì hàm trả về `"3"`. Đỏ.

Green:

```python
def fizzbuzz(n):
    if n % 3 == 0:
        return "Fizz"
    return str(n)
```

Chạy cả hai test: pass

Chạy cả hai test: pass.

Vòng 3

Red: thêm `assert fizzbuzz(5) == "Buzz"` → fail.
Green: thêm nhánh `if n % 5 == 0: return "Buzz"` → pass.

Vòng 4

Red: thêm `assert fizzbuzz(15) == "FizzBuzz"` → fail (vì đang trả về `"Fizz"`).

Green:

```python
def fizzbuzz(n):
    if n % 3 == 0 and n % 5 == 0:
        return "FizzBuzz"
    if n % 3 == 0:
        return "Fizz"
    if n % 5 == 0:
        return "Buzz"
    return str(n)
```

Tất cả pass.

Refactor — giờ thấy lặp lại `n % 3 == 0` và `n % 5 == 0`. Dọn lại:

```python
def fizzbuzz(n):
    ket_qua = ""
    if n % 3 == 0:
        ket_qua += "Fizz"
    if n % 5 == 0:
        ket_qua += "Buzz"
    return ket_qua or str(n)
```

Chạy lại toàn bộ test: vẫn xanh. Nhờ có test, bạn **yên tâm** rằng việc viết lại không làm hỏng gì.

Vài lời khuyên cho người mới

- **Bước nhỏ thôi.** Mỗi test chỉ kiểm tra một điều.
- **Đừng bỏ bước Red.** Nếu test pass ngay từ đầu, có thể test viết sai hoặc không kiểm tra gì cả.
- **Ở bước Green, đừng viết thừa.** Chỉ đủ để pass test hiện tại; tính năng khác để test sau "kéo" ra.
- **Chỉ refactor khi đang xanh**, và chạy lại test sau mỗi lần sửa.

Khi đã quen với `assert`, bạn có thể chuyển sang công cụ như `pytest` (Python) hay `unittest` cho tiện, nhưng ý tưởng Red – Green – Refactor vẫn y nguyên. Bạn muốn thử tự làm một bài tập nhỏ theo TDD không? Mình có thể đưa đề và review từng bước cho bạn.

Mình có 3 thắc mắc: (1) Bạn chưa nhắc đến việc lập danh sách các trường hợp cần test trước khi code. Kent Beck có mô tả bước này không? (2) Ở vòng 1, test fail vì NameError. Như vậy có tính là Red đúng nghĩa không, hay phải fail vì sai kết quả? (3) Từ vòng 2, các hàm test chỉ được định nghĩa mà không được gọi, vậy chúng có thực sự chạy không?

Recalling test-driven development history and sources.

Cả ba câu đều đúng chỗ. Câu (1) và (3) là thiếu sót của mình, câu (2) thì cần nói rõ hơn.

(1) Danh sách test: Kent Beck có mô tả, và đó là bước đầu tiên

Trong *Test-Driven Development: By Example* (2002), Beck luôn giữ một **to-do list** / **test list** bên cạnh khi code. Ông ghi ra các trường hợp nghĩ tới, gạch dần từng mục, và thêm mục mới khi phát hiện ra trong lúc làm. Năm 2023, trong bài "Canon TDD", ông nói rõ đây là **bước 1** của quy trình:

1. Lập danh sách các kịch bản cần test.
2. Chọn **đúng một** mục, biến nó thành một test cụ thể, chạy được.
3. Sửa code cho test đó (và mọi test trước) pass. Nếu nghĩ ra trường hợp mới, thêm vào danh sách.
4. Refactor nếu cần.
5. Quay lại bước 2 cho đến khi danh sách trống.

Như vậy Red – Green – Refactor là vòng lặp **bên trong**, còn danh sách test nằm **bên ngoài**. Danh sách này giúp bạn không phải nghĩ hết mọi thứ cùng lúc, và chọn được thứ tự hợp lý, thường là từ trường hợp đơn giản đến khó. Với FizzBuzz, lẽ ra mình nên bắt đầu như sau:

trường hợp đơn giản đến khó. Với FizzBuzz, ta có thể liệt kê như sau:

```
[ ] 1 -> "1"
[ ] 2 -> "2"
[ ] 3 -> "Fizz"
[ ] 6 -> "Fizz"        (bội của 3, không chỉ riêng số 3)
[ ] 5 -> "Buzz"
[ ] 10 -> "Buzz"
[ ] 15 -> "FizzBuzz"
```

(2) Fail vì NameError: được chấp nhận, nhưng chưa phải Red "tốt nhất"

Beck chấp nhận chuyện này. Trong sách, ví dụ bằng Java của ông thường có bước đỏ đầu tiên là **code không compile được**, vì class hay method chưa tồn tại. NameError trong Python là tình huống tương tự.

Tuy vậy, bạn nên để ý điểm này: mục đích của Red là xác nhận rằng **test có khả năng phát hiện sai**. NameError chỉ cho biết hàm chưa tồn tại, chứ chưa chứng minh câu `assert` hoạt động. Nếu bạn lỡ viết `assert fizzbuzz(1) == fizzbuzz(1)`, lỗi NameError vẫn xuất hiện y như vậy.

Vì vậy cách làm chặt chẽ hơn là thêm một bước trung gian:

python

```python
def fizzbuzz(n):
    pass          # stub: hàm tồn tại nhưng chưa làm gì
```

Chạy lại test, bạn sẽ thấy `AssertionError` (vì `None != "1"`). Đây mới là Red **vì đúng lý do**. Sau đó mới viết `return str(n)` để chuyển sang Green.

Quy tắc chung: khi test đỏ, hãy đọc thông báo lỗi và tự hỏi "có phải nó fail đúng vì lý do mình mong đợi không?"

(3) Đúng vậy: các test đó không hề chạy

Trong Python thuần, `def test_...()` chỉ **định nghĩa** hàm. Nếu không gọi, hàm sẽ không chạy. Vậy nên những chỗ mình viết "Chạy: fail" từ vòng 2 trở đi là sai: nếu bạn chạy file đó thật, chương trình sẽ im lặng và **trông như mọi thứ đều pass**. Đây chính là kiểu "xanh giả" mà TDD muốn tránh, và bước Red có thể giúp bạn bắt được lỗi này: nếu bạn mong đợi đỏ mà không thấy gì, chắc chắn có vấn đề.

(Lý do mình quen đặt tên `test_...` là vì `pytest` tự tìm và gọi các hàm có tiền tố đó. Không dùng pytest thì phải tự gọi.)

Dưới đây là file hoàn chỉnh, có gọi tất cả các test và có danh sách test ở đầu:

python

```python
# Danh sách test:
# [x] 1 -> "1"     [x] 2 -> "2"
# [x] 3 -> "Fizz"  [x] 6 -> "Fizz"
# [x] 5 -> "Buzz"  [x] 10 -> "Buzz"
# [x] 15 -> "FizzBuzz"

def fizzbuzz(n):
    ket_qua = ""
    if n % 3 == 0:
        ket_qua += "Fizz"
    if n % 5 == 0:
        ket_qua += "Buzz"
    return ket_qua or str(n)


def test_so_thuong():
    assert fizzbuzz(1) == "1"
    assert fizzbuzz(2) == "2"

def test_boi_cua_3():
    assert fizzbuzz(3) == "Fizz"
    assert fizzbuzz(6) == "Fizz"

def test_boi_cua_5():
    assert fizzbuzz(5) == "Buzz"
    assert fizzbuzz(10) == "Buzz"

def test_boi_cua_15():
    assert fizzbuzz(15) == "FizzBuzz"
```

```python
if __name__ == "__main__":
    tests = [test_so_thuong, test_boi_cua_3, test_boi_cua_5, test_boi_cua_15]
    for t in tests:
        t()                     # assert sai sẽ dừng ngay tại đây
        print("PASS", t.__name__)
    print("Tất cả test đều xanh.")
```

Mỗi khi thêm một test mới, nhớ thêm nó vào danh sách `tests`. Nếu quên, test đó sẽ không chạy, và bạn lại gặp đúng vấn đề ở câu (3). Đây cũng là một lý do thực tế khiến người ta chuyển sang dùng `pytest` khi đã nắm vững ý tưởng.

Mình sẽ xây dựng một Ticket Manager CLI có 4 lệnh: create, list, show, update. Dữ liệu lưu trong file JSON. Trước khi code, mình nên test những hành vi nào? Chia theo nhóm giúp mình.

Lập danh sách các trường hợp kiểm thử theo nhóm.

Trước khi lập danh sách test, bạn cần chốt vài quyết định, vì mỗi quyết định sẽ thay đổi câu chữ của test. Danh sách dưới đây giả định như sau (bạn sửa lại cho khớp với thiết kế của mình):

- Một ticket gồm: `id`, `title`, `description`, `status`, `created_at`.
- `id` là số nguyên tự tăng, bắt đầu từ 1.
- `status` chỉ nhận 3 giá trị `open`, `in_progress`, `done`. Ticket mới mặc định là `open`.
- `update` được sửa `title`, `description`, `status`. Không được sửa `id` và `created_at`.

Tách code thành 3 lớp để dễ test

Đây là gợi ý quan trọng nhất. Nếu mọi thứ nằm chung trong một hàm vừa đọc file, vừa xử lý, vừa in ra màn hình, bạn sẽ rất khó test. Hãy tách ra:

1. **Logic**: các hàm nhận và trả về dữ liệu Python (list, dict). Không đụng tới file, không `print`.
2. **Lưu trữ**: `load(path)` và `save(path, tickets)`. Nhận đường dẫn làm tham số, nhờ vậy khi test bạn có thể truyền file tạm thay cho file thật.
3. **CLI**: phân tích tham số dòng lệnh, gọi lớp 1 và 2, in kết quả. Lớp này càng mỏng càng tốt.

Các nhóm test bên dưới được sắp theo đúng 3 lớp này.

Nhóm A — `create` (logic)

```
[ ] Tạo ticket đầu tiên -> id = 1
[ ] Tạo ticket thứ hai -> id = 2
[ ] Ticket mới có status = "open"
[ ] Ticket mới có title đúng như đã truyền vào
[ ] description không truyền -> chuỗi rỗng (hoặc None, tùy bạn chọn)
[ ] created_at được gán giá trị
[ ] title rỗng hoặc chỉ toàn khoảng trắng -> báo lỗi
[ ] Xóa ticket id 2 (nếu sau này có) rồi tạo mới -> không dùng lại id 2
    (với 4 lệnh hiện tại chưa cần, nhưng nên nghĩ trước: id = max + 1 hay bộ đếm
riêng?)
```

Nhóm B — `list` (logic)

```
[ ] Chưa có ticket nào -> trả về danh sách rỗng
[ ] Có 3 ticket -> trả về đủ 3
[ ] Thứ tự trả về ổn định (ví dụ theo id tăng dần)
[ ] (nếu có) Lọc theo status -> chỉ trả về các ticket khớp
[ ] (nếu có) Lọc theo status không hợp lệ -> báo lỗi
```

Nhóm C — `show` (logic)

```
[ ] id tồn tại -> trả về đúng ticket đó
[ ] id không tồn tại -> báo lỗi "không tìm thấy"
[ ] Có nhiều ticket -> không trả nhầm ticket khác
```

Nhóm D — `update` (logic)

```
[ ] Sửa title -> title đổi, các trường khác giữ nguyên
[ ] Sửa status sang giá trị hợp lệ -> thành công
[ ] Sửa status sang giá trị không hợp lệ ("closed") -> báo lỗi, ticket KHÔNG bị đổi
[ ] Sửa nhiều trường cùng lúc -> tất cả đều đổi
[ ] Không truyền trường nào để sửa -> báo lỗi (hoặc không làm gì, tùy bạn chọn)
```

```
[ ] id không tồn tại -> báo lỗi
[ ] Sửa title thành rỗng -> báo lỗi (cùng quy tắc với create)
[ ] Không cho sửa id và created_at
```

Lưu ý dòng thứ ba: khi gặp lỗi, ngoài việc kiểm tra là có báo lỗi, bạn nên kiểm tra thêm là **dữ liệu không bị thay đổi một nửa**. Lỗi kiểu này rất dễ bỏ sót.

Nhóm E — Lưu trữ JSON

```
[ ] save rồi load -> nhận lại đúng dữ liệu (round-trip)
[ ] File chưa tồn tại -> load trả về danh sách rỗng, không crash
[ ] File rỗng (0 byte) -> xử lý thế nào? (rỗng hay báo lỗi — cần chốt)
[ ] File JSON hỏng -> báo lỗi rõ ràng, KHÔNG ghi đè làm mất dữ liệu
[ ] Tiếng Việt có dấu trong title -> lưu và đọc lại đúng (ensure_ascii, encoding
utf-8)
[ ] Sau khi create, chạy lại chương trình (load lại) -> ticket vẫn còn
[ ] id tiếp tục tăng đúng sau khi load lại (không quay về 1)
```

Hai dòng cuối là những lỗi hay gặp nhất ở loại ứng dụng này: test trong bộ nhớ thì pass, nhưng khi chạy lại chương trình thì id bắt đầu lại từ 1.

Nhóm F — Giao diện CLI

```
[ ] create --title "..." -> in ra id vừa tạo, exit code 0
[ ] list -> in ra mỗi ticket một dòng
[ ] list khi rỗng -> in thông báo thân thiện ("Chưa có ticket nào")
[ ] show 1 -> in đủ các trường
[ ] show 99 -> in lỗi ra stderr, exit code khác 0
[ ] show abc (id không phải số) -> báo lỗi, không crash với traceback
[ ] Lệnh không tồn tại (delete) -> in hướng dẫn sử dụng, exit code khác 0
[ ] Thiếu tham số bắt buộc (create không có --title) -> báo lỗi
```

Nhóm này nên ít test hơn các nhóm khác, vì phần logic đã được kiểm tra ở nhóm A–D. Ở đây bạn chỉ cần kiểm tra là CLI "nối dây" đúng và xử lý lỗi nhập liệu.

Thứ tự nên làm

Theo cách của Beck, bạn bắt đầu từ trường hợp đơn giản nhất và mở rộng dần:

1. `list` khi rỗng (B1). Đây là test dễ nhất, giúp bạn dựng xong bộ khung.
2. `create` một ticket, rồi `list` thấy nó (A1, B2).
3. Các trường hợp còn lại của A, C, D, mỗi lần một test.
4. Nhóm E, chạy với file tạm (`tempfile` trong thư viện chuẩn của Python).
5. Nhóm F cuối cùng.

Trong lúc làm, chắc chắn bạn sẽ nghĩ ra thêm trường hợp mới. Cứ thêm vào danh sách, đừng làm ngay. Đó chính là cách dùng test list như Beck mô tả.

Nếu bạn muốn, mình có thể cùng bạn làm vòng Red – Green – Refactor đầu tiên cho mục B1, kèm cấu trúc thư mục và cách dùng file tạm.

Mình dùng Node.js chứ không phải Python. Danh sách còn vài chỗ thiếu: (1) Có nên test giới hạn độ dài tiêu đề không, và nên test những giá trị biên nào? (2) Nếu file JSON hợp lệ nhưng không phải mảng thì sao? (3) Lỗi cú pháp lệnh (lệnh không tồn tại, thiếu tham số) và lỗi nghiệp vụ (không tìm thấy ticket, status sai) có nên dùng exit code khác nhau không? (4) Ngoài "abc", lệnh `show` nên từ chối những dạng id không hợp lệ nào nữa?

Xem xét giới hạn độ dài, định dạng JSON và mã thoát chương trình. ›

Mình chuyển sang Node.js. Mấy chỗ mình nói theo kiểu Python ở trên có tương đương trong Node như sau: file tạm thì dùng `fs.mkdtempSync(path.join(os.tmpdir(), 'tickets-'))`, đọc ghi tiếng Việt thì nhớ truyền `'utf8'` cho `readFileSync` và `writeFileSync`, còn phân tích tham số dòng lệnh thì có sẵn `util.parseArgs` trong thư viện chuẩn, không cần cài gì thêm.

(1) Giới hạn độ dài tiêu đề

Có, nên test, nếu bạn đặt ra giới hạn. Mà nên đặt, vì không có giới hạn thì người dùng có thể dán cả trang văn bản vào, và lệnh `list` sẽ vỡ hết bố cục. Giả sử giới hạn là 1–200 ký tự, kỹ thuật **giá trị biên** (boundary value) bảo bạn test ngay tại hai mép và sát ngoài mép:

```
[ ] ""            -> lỗi   (dưới mức tối thiểu)
[ ] "a"           -> OK    (đúng mức tối thiểu)
[ ] 200 ký tự     -> OK    (đúng mức tối đa)
```

```
[ ] 201 ký tự      -> lỗi   (vượt mức tối đa)
```

Ngoài 4 test trên, có hai câu hỏi bạn phải tự quyết định, và mỗi quyết định nên có test đi kèm:

Có cắt khoảng trắng hai đầu (trim) trước khi đếm không?

```
[ ] "   "                    -> lỗi (trim xong thành rỗng)
[ ] "  abc  "                -> lưu thành "abc"
[ ] 200 ký tự + 2 dấu cách   -> OK nếu trim trước khi đếm
```

"Ký tự" được đếm theo đơn vị nào? Đây là bẫy riêng của JavaScript. `str.length` đếm theo đơn vị UTF-16, không phải theo số ký tự mắt người nhìn thấy:

```
js

"😜".length          // 2
"ê".length           // 1 nếu ở dạng NFC, 2 nếu ở dạng NFD
```

Tiếng Việt gõ trên macOS hoặc dán từ một số nguồn có thể ở dạng NFD (chữ cái và dấu tách rời nhau). Cách xử lý gọn nhất là chuẩn hóa `title.normalize('NFC')` trước khi lưu, rồi đếm bằng `[...title].length`. Các test nên có:

```
[ ] 200 chữ "ê" ở dạng NFD -> OK (sau khi chuẩn hóa vẫn là 200)
[ ] 200 emoji              -> OK nếu đếm theo code point
[ ] title có ký tự xuống dòng "\n" -> lỗi (không thì list sẽ vỡ dòng)
```

(2) File JSON hợp lệ nhưng không phải mảng

`JSON.parse` sẽ chấp nhận bất kỳ giá trị JSON nào. Nếu bạn chỉ bắt `SyntaxError` thì những trường hợp sau sẽ lọt qua và làm crash ở chỗ khác:

```
[ ] {}        -> lỗi "file dữ liệu không đúng định dạng"
[ ] null      -> lỗi
[ ] "abc"     -> lỗi
[ ] 42        -> lỗi
[ ] true      -> lỗi
```

Nên kiểm tra bằng `Array.isArray(data)`. Đừng dùng `typeof`, vì `typeof [] === 'object'` và `typeof null === 'object'`.

Kế đến là trường hợp **file là mảng nhưng phần tử bên trong sai**. Bạn cần quyết định sẽ kiểm tra sâu đến mức nào:

```
[ ] [1, 2, 3]                      -> lỗi (phần tử không phải object)
[ ] [{"title": "x"}]               -> lỗi (thiếu id)
[ ] [{"id": 1}, {"id": 1}]         -> lỗi (trùng id)
[ ] [{"id": "1", ...}]             -> lỗi (id không phải số)
[ ] [{"id": 1, "status": "closed"}] -> lỗi (status lạ)
```

Thêm một trường hợp hay gặp: file có **BOM** ở đầu, chẳng hạn sau khi mở và lưu bằng Notepad trên Windows. Lúc này `JSON.parse` sẽ báo lỗi dù nội dung nhìn hoàn toàn bình thường. Bạn có thể bỏ BOM trước khi parse, hoặc báo lỗi rõ ràng. Chọn cách nào cũng được, miễn là có test cho nó.

Và quy tắc quan trọng nhất cho cả nhóm này: **gặp file hỏng thì không bao giờ ghi đè**. Test như sau: tạo file hỏng, chạy `create`, rồi kiểm tra nội dung file vẫn giữ nguyên từng byte.

Ghi chú về thiết kế: nếu bạn muốn id không bị dùng lại sau này, định dạng `{ "nextId": 4, "tickets": [...] }` sẽ tiện hơn một mảng trần. Khi đó, mọi thứ không có dạng object như vậy đều bị coi là file hỏng. Bạn nên chốt định dạng trước khi viết test cho nhóm này.

(3) Exit code: có nên tách lỗi cú pháp và lỗi nghiệp vụ không?

Nên tách. Người viết script gọi CLI của bạn sẽ cần phân biệt "mình gọi sai lệnh" với "lệnh đúng nhưng dữ liệu không như mong đợi". Không có chuẩn bắt buộc nào ở đây, nhưng quy ước sau được dùng phổ biến:

Exit code	Ý nghĩa	Ví dụ
0	Thành công	
1	Lỗi nghiệp vụ	`show 99` không tìm thấy
2	Lỗi cách dùng (usage)	lệnh lạ, thiếu `--title`, id `abc`
3	Lỗi dữ liệu lưu trữ (tùy chọn)	file JSON hỏng

Để không phải phân vân với từng trường hợp, hãy dùng quy tắc này: **nếu chỉ nhìn tham số mà không cần đọc file đã biết là sai, đó là lỗi usage (2). Nếu phải đọc dữ liệu mới biết, đó là lỗi nghiệp vụ (1).** Theo quy tắc đó, `update 1 --status closed` là lỗi 2 (biết ngay `closed` không

nghiệp vụ (1). Theo quy tắc đó, `update 1 --status closed` là lỗi 2 (biết ngày `closed` không hợp lệ), còn `show 99` là lỗi 1 (phải đọc file mới biết là không có).

Các test cho nhóm này nên kiểm tra **cả ba thứ**: exit code, stdout và stderr:

```
[ ] Lỗi in ra stderr, stdout để trống
[ ] Thành công in ra stdout, stderr để trống
[ ] Mỗi loại lỗi trả về đúng exit code đã ghi trong bảng
```

Trong Node, bạn chạy CLI như một tiến trình thật để kiểm tra:

```js
const { spawnSync } = require('node:child_process');
const r = spawnSync('node', ['cli.js', 'show', '99'], { encoding: 'utf8' });
// r.status, r.stdout, r.stderr
```

Lưu ý khi viết CLI: nên đặt `process.exitCode = 1` thay vì gọi `process.exit(1)`. Gọi `process.exit()` có thể cắt mất phần output chưa kịp ghi ra khi stdout là pipe, và khi đó test của bạn sẽ chập chờn lúc pass lúc fail.

(4) Những dạng id không hợp lệ cần từ chối

Phần này đặc biệt quan trọng trong JavaScript, vì `Number()` và `parseInt()` dễ dãi hơn bạn tưởng:

```js
parseInt("12abc")  // 12   <- lọt qua
parseInt("1.5")    // 1    <- lọt qua
Number("")         // 0
Number(" 12 ")     // 12
Number("0x1A")     // 26
Number("1e2")      // 100
```

Danh sách nên từ chối (exit code 2):

```
[ ] ""            chuỗi rỗng
[ ] " "           chỉ có khoảng trắng
[ ] " 12 "        khoảng trắng hai đầu
[ ] "0"           id bắt đầu từ 1
[ ] "-1"          số âm (lưu ý: parseArgs có thể hiểu nhầm là một option)
[ ] "+1"          có dấu cộng
[ ] "1.5"         số thập phân
[ ] "1.0"         thập phân dù giá trị là số nguyên
[ ] "12abc"       số kèm chữ
[ ] "0x1A"        hệ 16
[ ] "1e2"         ký pháp khoa học
[ ] "01"          số 0 ở đầu (tùy bạn chọn, nhưng phải nhất quán)
[ ] "Infinity", "NaN"
[ ] "9999999999999999999"  vượt Number.MAX_SAFE_INTEGER
[ ] thiếu id      `show` không có tham số
[ ] thừa tham số  `show 1 2`
```

Và đừng quên test chiều ngược lại, để chắc là bạn không chặn quá tay:

```
[ ] "1"  -> hợp lệ
[ ] "42" -> hợp lệ
```

Cách kiểm tra gọn và dễ đọc nhất là dùng regex kết hợp với kiểm tra số nguyên an toàn:

```js
function parseId(raw) {
  if (!/^[1-9]\d*$/.test(raw)) return null;
  const n = Number(raw);
  return Number.isSafeInteger(n) ? n : null;
}
```

Hàm `parseId` này thuộc lớp logic, nên bạn có thể test nó trực tiếp mà không cần chạy CLI. Đó là cách hay: dồn hết 18 trường hợp trên vào test của `parseId`, rồi ở tầng CLI chỉ cần một hai test để xác nhận `show abc` trả về exit code 2. Phần lớn công sức test nằm ở hàm nhỏ, tầng CLI chỉ cần kiểm tra là đã gọi đúng hàm.

Viết cho mình một unit test đang fail cho trường hợp tạo ticket với tiêu đề rỗng, rồi

Viết cho mình một unit test đang fail cho trường hợp tạo ticket với tiêu đề rỗng, rồi viết code tối thiểu để test pass. Dùng test runner có sẵn của Node.js (node:test).

Alegând tactica minimă cinstită de tip „fake it". >

Mình sẽ viết test, chạy thật để thấy nó đỏ đúng lý do, sau đó viết code tối thiểu và chạy lại cho xanh.

Ran 4 commands >

Mình đã chạy thật cả ba bước. Dưới đây là kết quả từng bước.

Cấu trúc

```
ticket-manager/
├── package.json        "test": "node --test"
├── src/tickets.js
└── test/tickets.test.js
```

`node --test` tự tìm các file `*.test.js`, nên bạn không cần cài thêm gì.

Bước 1 — Red: viết test trước

```js
// test/tickets.test.js
const { test } = require('node:test');
const assert = require('node:assert/strict');
const { createTicket } = require('../src/tickets');

test('create: tiêu đề rỗng thì báo lỗi EMPTY_TITLE', () => {
  assert.throws(
    () => createTicket([], { title: '' }),
    { code: 'EMPTY_TITLE' }
  );
});
```

Chạy `npm test`, kết quả là `Cannot find module '../src/tickets'`. Test đỏ, nhưng như đã bàn ở câu (2) lúc trước, **đây chưa phải đỏ đúng lý do**: lỗi này chỉ cho biết file chưa tồn tại.

Bước 1b — Thêm stub để thấy đỏ đúng lý do

```js
// src/tickets.js
function createTicket(tickets, { title }) {
  // stub: hàm đã tồn tại nhưng chưa làm gì
}

module.exports = { createTicket };
```

Chạy lại, kết quả là `Missing expected exception.` Giờ thì đúng rồi: hàm đã chạy, nhưng không ném lỗi như test yêu cầu. Điều này chứng tỏ `assert.throws` thực sự kiểm tra được hành vi.

Bước 2 — Green: code tối thiểu

```js
// src/tickets.js
function createTicket(tickets, { title }) {
  if (title === '') {
    const err = new Error('Tiêu đề không được để trống');
    err.code = 'EMPTY_TITLE';
    throw err;
  }
}

module.exports = { createTicket };
```

```
ok 1 - create: tiêu đề rỗng thì báo lỗi EMPTY_TITLE
# pass 1
# fail 0
```

Vài điểm đáng chú ý

Vì sao kiểm tra `code` thay vì kiểm tra câu thông báo? Câu thông báo là thứ hiển thị cho người dùng, bạn có thể đổi cách viết bất cứ lúc nào. `code` là mã ổn định dành cho máy đọc. Sau này tầng CLI cũng sẽ dựa vào `code` để chọn exit code (`EMPTY_TITLE` thuộc lỗi usage, tức exit code 2). Nếu test dựa vào câu chữ, chỉ cần sửa một dấu câu là test đỏ, dù hành vi không hề đổi.

Code này "ngây thơ" có chủ ý. Nó dùng `title === ''`, nên `'   '` vẫn lọt qua. Hàm cũng chưa trả về ticket nào khi title hợp lệ. Đó là đúng tinh thần Green: chỉ viết đủ cho test hiện tại. Những hành vi còn lại sẽ do các test tiếp theo trong danh sách "kéo" ra:

```
[x] title ""          -> lỗi EMPTY_TITLE
[ ] title "   "       -> lỗi EMPTY_TITLE      <- sẽ buộc phải thêm trim()
[ ] title "Sửa lỗi"   -> trả về ticket id 1   <- sẽ buộc hàm phải trả về giá trị
[ ] title rỗng        -> mảng tickets không bị thay đổi
```

Refactor: code hiện tại mới có ba dòng, chưa có gì để dọn, nên bỏ qua bước này ở vòng đầu tiên.

Theo mình, test nên làm tiếp là `"   "`. Nó chỉ cần sửa đúng một dòng và cho bạn một vòng Red – Green rất gọn để luyện tay.

Với danh sách test hiện tại, còn những edge case nào về input sai, ticket không tồn tại và file JSON hỏng mà mình đang bỏ sót? Và làm sao để biết bộ test của mình đã đủ mạnh?

Xem xét các trường hợp dữ liệu JSON hỏng và cách kiểm tra độ bao phủ mạnh. ›

Mình kiểm tra nhanh vài hành vi của Node trước khi liệt kê, để những gì nói dưới đây đều đúng thực tế.

Probe current code and Node behaviors for edge cases ›

Kết quả kiểm tra có một điều rất đáng chú ý: **code hiện tại đạt 100% coverage, nhưng `createTicket([], {})` vẫn tạo được ticket không có tiêu đề.** Mình sẽ quay lại chi tiết này ở phần 2, vì nó trả lời luôn câu hỏi "làm sao biết bộ test đủ mạnh".

Phần 1 — Các edge case còn thiếu

Input sai

Sai kiểu dữ liệu. Hiện tại code chỉ chặn đúng chuỗi `''`, nên những trường hợp sau đều lọt qua (mình đã chạy thử):

```
[ ] createTicket([], {})               -> title undefined: đang KHÔNG báo lỗi
[ ] createTicket([], { title: null })  -> đang KHÔNG báo lỗi
[ ] createTicket([], { title: 123 })   -> số thay vì chuỗi
[ ] createTicket([], { title: ['a'] }) -> mảng
[ ] createTicket([])                   -> đang crash bằng TypeError, không phải lỗi
có code
```

Ký tự "trông như rỗng". `trim()` không xóa ký tự zero-width (`'\u200B'.trim()` vẫn dài 1), nên tiêu đề nhìn trống trơn vẫn được chấp nhận:

```
[ ] "\u200B"           -> nên báo EMPTY_TITLE
[ ] "\t"               -> tab, trim xóa được, nhưng nên có test
[ ] "abc\u0000"        -> ký tự điều khiển
```

Tham số dòng lệnh (với `util.parseArgs`):

```
[ ] --title -urgent      -> parseArgs từ chối (ERR_PARSE_ARGS_INVALID_OPTION_VALUE).
                            Phải dùng --title=-urgent. Cần test cả hai cách.
[ ] --title a --title b  -> parseArgs lấy "b" một cách âm thầm. Chấp nhận hay báo
lỗi?
[ ] --title=             -> giá trị rỗng qua cú pháp dấu bằng
[ ] --priority high      -> option lạ, parseArgs ném ERR_PARSE_ARGS_UNKNOWN_OPTION.
                            Bạn phải bắt lỗi này và đổi thành exit code 2, không để
in traceback.
```

Quy tắc nghiệp vụ của `update` (bạn cần quyết định trước, rồi viết test theo quyết định đó):

```
[ ] --status OPEN / Open / " open"  -> phân biệt hoa thường không?
[ ] update status open -> open       -> thành công hay báo "không có gì thay đổi"?
[ ] done -> open                     -> có cho mở lại ticket đã xong không?
[ ] update --id 5 / --created_at ... -> phải báo lỗi, không âm thầm bỏ qua
```

Ticket không tồn tại

Ticket không tồn tại

```
[ ] show 1 khi file chưa tồn tại     -> lỗi "không tìm thấy" (exit 1), không crash
[ ] update 1 khi file chưa tồn tại   -> báo lỗi VÀ không tự tạo file mới
[ ] update 99 (không tìm thấy)       -> nội dung file giữ nguyên từng byte
[ ] show 9007199254740991            -> id hợp lệ nhưng không có: exit 1, không phải
exit 2
[ ] file chứa "id": "1" (chuỗi, do sửa tay) -> nếu so sánh bằng ===, show 1 sẽ báo
                                        "không tìm thấy" sai. Lớp kiểm tra file phải
bắt được lỗi này trước.
```

Dòng thứ ba là lỗi dễ bỏ sót nhất: code đọc file, không tìm thấy ticket, nhưng vẫn gọi `save()` ở cuối hàm và ghi lại file (có thể với định dạng khác).

File JSON hỏng

Những gì đã có: lỗi cú pháp, không phải mảng, BOM, phần tử sai, id trùng. Những gì còn thiếu:

```
[ ] File chỉ có khoảng trắng / xuống dòng
[ ] JSON bị cắt ngang giữa chừng  {"id": 1, "tit
[ ] JSON có comment hoặc dấu phẩy thừa (người dùng tự sửa tay)
[ ] Đường dẫn là một thư mục, không phải file     -> EISDIR
[ ] File không có quyền đọc                        -> EACCES
[ ] Thư mục không có quyền ghi khi save            -> báo lỗi rõ ràng
[ ] nextId nhỏ hơn hoặc bằng id lớn nhất (nếu dùng định dạng {nextId, tickets})
                                                   -> lần create sau sẽ tạo id trùng
[ ] created_at không phải ngày hợp lệ
[ ] Ticket có trường lạ (ví dụ "priority")         -> giữ lại hay xóa khi save?
```

Trong số này, **"JSON bị cắt ngang"** đáng để bạn nghĩ kỹ nhất, vì nó cho thấy một lỗi thiết kế chứ không chỉ là lỗi input. Nếu `save()` ghi đè trực tiếp lên file và chương trình bị tắt giữa chừng, toàn bộ dữ liệu sẽ mất. Cách phòng là ghi ra file tạm rồi `fs.renameSync()` đè lên file gốc. Test cho trường hợp này: giả lập ghi thất bại (ví dụ truyền vào một hàm ghi luôn ném lỗi), rồi kiểm tra file gốc vẫn còn nguyên.

Còn một trường hợp bạn nên **chủ động ghi rõ là không xử lý**: hai lệnh chạy cùng lúc, cả hai đọc file, rồi cả hai ghi lại, và một thay đổi bị mất. Với một CLI cá nhân, chấp nhận hạn chế này là hợp lý, nhưng hãy ghi nó vào README để người khác biết đó là lựa chọn có chủ đích chứ không phải bị bỏ sót.

Phần 2 — Làm sao biết bộ test đủ mạnh?

Coverage cho biết chỗ chưa test, nhưng không cho biết test đã mạnh

```
node --test --experimental-test-coverage
```

Lệnh này báo `tickets.js` đạt **100%** dòng và nhánh. Vậy mà như đã thấy ở trên, `title: undefined` vẫn lọt qua. Coverage chỉ đo "dòng code có được chạy hay không", không đo "nếu dòng đó sai thì có test nào phát hiện không". Nên đọc coverage theo một chiều: **dưới 100% chắc chắn là có chỗ chưa test, còn 100% thì chưa chứng minh được gì.**

Mutation testing: cố tình làm hỏng code

Đây là cách kiểm tra trực tiếp nhất. Bạn sửa sai một chỗ trong code, chạy test, và **mong đợi test đỏ**. Nếu test vẫn xanh, bạn vừa tìm ra chỗ yếu. Bạn có thể tự làm bằng tay, không cần công cụ:

Sửa code thành	Test nào phải đỏ?
Xóa `.trim()`	test `"  "`
`length > 200` đổi thành `length >= 200`	test 200 ký tự
`length > 200` đổi thành `length > 201`	test 201 ký tự
Bỏ dòng kiểm tra `Array.isArray`	test file chứa `{}`
Gọi `save()` cả khi không tìm thấy ticket	test "file giữ nguyên khi update 99"
`process.exitCode = 2` đổi thành `1`	test exit code của `show abc`

Hai dòng về giới hạn 200 cho thấy vì sao cần test cả hai phía biên: thiếu một trong hai, sẽ có một kiểu sửa sai không bị phát hiện. Khi đã quen, có công cụ tự động làm việc này cho JavaScript là **Stryker**, nhưng làm bằng tay vài lần sẽ giúp bạn hiểu bản chất trước.

Các dấu hiệu khác

- **Mọi test đều đã từng đỏ đúng lý do.** Nếu bạn làm TDD nghiêm túc, điều này tự động đúng. Còn test viết sau khi có code thì hãy tự làm hỏng code một lần để xem nó đỏ.
- **Kiểm tra cả tác dụng phụ, không chỉ giá trị trả về.** Khi có lỗi: dữ liệu có bị đổi một nửa không, file có bị ghi không, stdout có sạch không?
- **Mỗi bug tìm được đều bắt đầu bằng một test đỏ.** Trước khi sửa `title: undefined`, hãy viết

test tại niện lỗi đó. Bug sẽ không quay lại mà không bị phát hiện.

- **Rà lại theo một bộ câu hỏi cố định** cho mỗi input: rỗng / một / nhiều? Đúng biên / sát ngoài biên? Sai kiểu? Thiếu? Trùng? Thất bại giữa chừng thì sao?

Bước tiếp theo hợp lý nhất là sửa đúng lỗi mình vừa tìm ra: viết test `createTicket([], {})` (chắc chắn sẽ đỏ), rồi thay `title === ''` bằng một kiểm tra chặn được cả `undefined`, `null` và kiểu không phải chuỗi.

> Ticket CLI của mình có 3 lớp: CLI (phân tích lệnh, in kết quả), logic xử lý ticket, và lớp lưu trữ đọc/ghi file JSON. Mình muốn test chạy nhanh, không đụng vào file dữ liệu thật trên máy, nhưng vẫn chắc chắn phần đọc/ghi file hoạt động đúng khi file bị hỏng. Khi chọn cách test cho lớp lưu trữ, mình cần cân nhắc những vấn đề gì? Xếp theo mức độ quan trọng.

Check chmod behavior as root, EISDIR, and temp-dir speed ›

Mình đã đo thử vài điều trên máy này, kết quả được dùng ở các mục bên dưới. Các vấn đề cần cân nhắc, xếp từ quan trọng nhất trở xuống:

1. An toàn: test không bao giờ được chạm vào file thật

Trong tất cả các lỗi có thể xảy ra, đây là lỗi duy nhất **làm mất dữ liệu của chính bạn**. Các lỗi khác chỉ khiến test sai.

Rủi ro thường đến từ một giá trị mặc định nằm sâu trong code, kiểu như `load(path = '~/.tickets.json')`. Chỉ cần một test quên truyền đường dẫn là nó sẽ ghi đè lên dữ liệu thật. Cách phòng:

- Lớp lưu trữ **bắt buộc** nhận đường dẫn qua tham số, không có giá trị mặc định.
- Đường dẫn mặc định chỉ được tính ở một nơi duy nhất là điểm vào của CLI, và có thể ghi đè bằng biến môi trường (ví dụ `TICKETS_FILE`). Test CLI luôn đặt biến này trỏ tới file tạm.

```js
// storage.js: không có đường dẫn mặc định
function load(filePath) { ... }
function save(filePath, data) { ... }

// cli.js: nơi duy nhất biết đường dẫn thật
const filePath = process.env.TICKETS_FILE ?? path.join(os.homedir(), '.tickets.json');
```

2. Dùng file system thật, đừng mock `fs`

Đây là điểm dễ chọn sai nhất. Mục tiêu của bạn là "chắc chắn đọc/ghi đúng khi file hỏng". Nếu mock `fs`, bạn đang test **giả định của mình về** `fs`, chứ không phải hành vi thật của nó. Các trường hợp như BOM, EISDIR, mã hóa UTF-8, hay việc `rename` đè lên file có sẵn chỉ được kiểm chứng thật khi chạy trên đĩa thật.

Lo ngại về tốc độ thì không có cơ sở: mình đo được **100 lần tạo thư mục tạm, ghi, đọc rồi xóa chỉ mất khoảng 34 ms**, tức chưa tới nửa mili giây mỗi lần.

Chỉ nên dùng giả lập cho những lỗi không thể tạo ra một cách ổn định trên đĩa thật (xem mục 5).

3. Tốc độ đến từ cách chia lớp, không đến từ việc mock

Bạn muốn test chạy nhanh, và câu trả lời nằm ở chỗ **đặt phần lớn test vào đâu**:

Lớp	Số lượng test	Đụng tới đĩa?
Logic	Nhiều nhất (hàng chục)	Không. Chỉ làm việc với mảng và object
Lưu trữ	Vừa phải	Có, nhưng chỉ trong thư mục tạm
CLI	Ít, mỗi lệnh vài test	Có, chạy tiến trình thật bằng `spawnSync`

Nếu thấy mình viết nhiều test lưu trữ để kiểm tra những chuyện như "title rỗng", đó là dấu hiệu logic đang bị trộn lẫn vào lớp lưu trữ. Lớp lưu trữ chỉ nên trả lời một câu: **đọc ra đúng, ghi vào đúng, gặp file hỏng thì báo đúng lỗi.**

4. Mỗi test có thư mục tạm riêng và luôn được dọn dẹp

`node --test` chạy các file test **song song, mỗi file một tiến trình**. Nếu hai test cùng dùng `os.tmpdir()/tickets.json`, chúng sẽ ghi đè lên nhau và test sẽ đỏ ngẫu nhiên. Đây là loại lỗi rất khó tìm.

```js
const { test } = require('node:test');
```

```
const fs = require('node:fs');
const os = require('node:os');
const path = require('node:path');

function tempFile(t, content) {
  const dir = fs.mkdtempSync(path.join(os.tmpdir(), 'tickets-'));
  t.after(() => fs.rmSync(dir, { recursive: true, force: true }));
  const file = path.join(dir, 'tickets.json');
  if (content !== undefined) fs.writeFileSync(file, content);
  return file;
}

test('load: file chứa {} thì báo lỗi CORRUPT_DATA', (t) => {
  const file = tempFile(t, '{}');
  // ...
});
```

`t.after` vẫn chạy dù test bị đỏ, nên thư mục tạm luôn được xóa.

5. Tạo lỗi phải ổn định trên mọi máy

Có ba mức độ, từ dễ đến khó:

Nội dung hỏng: rất dễ, chỉ cần ghi đúng những byte bạn muốn (`'{}'`, `'\uFEFF[]'`, `'{"id": 1, "tit'`). Chạy giống nhau ở mọi nơi.

Lỗi hệ thống tạo ra được bằng tình huống thật: truyền vào đường dẫn của một thư mục thay vì file sẽ cho ra `EISDIR` trên mọi hệ điều hành. Nên ưu tiên cách này.

Lỗi phụ thuộc môi trường: cẩn thận. Mình đã thử `chmod 000` một file rồi đọc, và **vẫn đọc được**, vì máy này đang chạy với quyền root. Docker và nhiều hệ thống CI cũng chạy bằng root, còn trên Windows thì `chmod` gần như không có tác dụng. Một test kiểu "không có quyền đọc thì báo lỗi" sẽ đỏ trên CI mà bạn không hiểu vì sao. Có hai cách xử lý:

- Bỏ qua có điều kiện và ghi rõ lý do: `test('...', { skip: process.getuid?.() === 0 && 'root bỏ qua quyền file' }, ...)`.
- Hoặc với các lỗi như ghi thất bại giữa chừng hay đĩa đầy, cho `save` nhận hàm ghi qua tham số, rồi trong test truyền vào một hàm luôn ném lỗi. Đây là chỗ duy nhất nên dùng giả lập.

6. Kiểm tra đến từng byte, không chỉ kiểm tra là có báo lỗi

Với file hỏng, hành vi quan trọng nhất là **không làm hỏng thêm**. Nên các test cần khẳng định cả những gì *không* xảy ra:

```
js

const before = fs.readFileSync(file);           // Buffer
assert.throws(() => updateCommand(file, ...), { code: 'CORRUPT_DATA' });
assert.ok(before.equals(fs.readFileSync(file))); // file giữ nguyên từng byte
assert.deepEqual(fs.readdirSync(dir), ['tickets.json']); // không còn file tạm sót lại
```

Dòng cuối dành cho trường hợp bạn ghi kiểu nguyên tử (ghi ra file tạm rồi `rename`): khi ghi thất bại, file tạm cũng phải được dọn đi.

7. Hợp đồng lỗi giữa các lớp

Lớp CLI quyết định exit code dựa trên lỗi mà lớp lưu trữ ném ra. Hãy cố định điều này bằng test ở cả hai phía:

- Test lớp lưu trữ: file hỏng thì ném lỗi với `code: 'CORRUPT_DATA'`, không phải `SyntaxError` gốc của `JSON.parse`.
- Test CLI: gặp file hỏng thì thoát với exit code 3, thông báo in ra stderr.

Nếu bỏ qua bước này, một ngày nào đó có người đổi tên mã lỗi, test của từng lớp vẫn xanh, nhưng CLI lại in ra traceback.

8. Khác biệt giữa các hệ điều hành (ưu tiên thấp)

Chỉ quan trọng nếu có người dùng chạy trên Windows: file sửa bằng tay có thể dùng xuống dòng kiểu CRLF, Windows không cho `rename` đè lên một file đang mở, và nên dùng `path.join` thay vì nối chuỗi bằng `/`. Hãy ghi những điểm này vào danh sách test, nhưng chưa cần làm ngay.

Tóm lại, với lớp lưu trữ, cách hợp lý là **file thật trong thư mục tạm riêng cho từng test, đường dẫn luôn được truyền vào, và chỉ giả lập những lỗi không thể tạo ra một cách ổn định**. Theo mình, nên bắt đầu bằng helper `tempFile` ở mục 4 cùng một test round-trip `save` rồi `load`. Hai thứ đó là nền cho mọi test còn lại của lớp này.

Bạn đã đưa ra kết luận ngay từ đầu. Mình muốn so sánh công bằng trước khi chọn: hãy liệt kê tất cả các phương án test lớp lưu trữ (kể cả mock fs, fake trong bộ nhớ, file thật trong thư mục tạm, và các cách khác nếu có), kèm ưu điểm và nhược điểm thật sự của từng cách.

Cataloging dependency injection, fakes, and in-memory options. ›

Bạn nói đúng: ở câu trước mình đã chốt "dùng file thật, đừng mock `fs`" trước khi so sánh, và câu đó nói quá tuyệt đối. Lần này mình sẽ bày hết các phương án ra. Trước tiên mình kiểm tra hai chi tiết về khả năng mock có sẵn trong Node 22, vì chúng ảnh hưởng trực tiếp đến đánh giá.

Test node:test fs mocking behavior and module mock flag ›

Mình đã xác nhận hai điều trên Node 22 của máy này, cả hai đều ảnh hưởng đến đánh giá bên dưới:

- `mock.method(fs, 'readFileSync', ...)` **chỉ có tác dụng nếu code gọi** `fs.readFileSync(...)`. Nếu code viết `const { readFileSync } = require('node:fs')`, mock bị bỏ qua hoàn toàn và code chạm vào đĩa thật (test của mình nhận về `ENOENT`).
- `mock.module` (thay cả module) **chưa có sẵn**, phải bật cờ `--experimental-test-module-mocks`.

Tiêu chí so sánh

Mỗi phương án được đánh giá theo 6 tiêu chí:

1. **Độ trung thực:** test có phản ánh hành vi thật của hệ điều hành không?
2. **An toàn:** có thể lỡ chạm vào file thật không?
3. **Tạo lỗi:** có dễ giả lập file hỏng, lỗi quyền, đĩa đầy, sập giữa chừng không?
4. **Độ gắn với cách cài đặt:** refactor mà không đổi hành vi thì test có bị đỏ không?
5. **Tốc độ.**
6. **Phụ thuộc thêm** và khả năng chạy được trên nhiều hệ điều hành.

A. File thật trong thư mục tạm

Cách làm: mỗi test tạo thư mục bằng `fs.mkdtempSync`, ghi nội dung cần thiết, gọi `load` / `save` thật, rồi xóa thư mục.

Ưu điểm

- Trung thực nhất: BOM, mã hóa UTF-8, `EISDIR`, `rename` đè file đều là hành vi thật.
- Không gắn với cách cài đặt: bạn đổi `readFileSync` thành `fs.promises.readFile`, test vẫn xanh.
- Nhanh hơn mức người ta thường nghĩ (khoảng 0,3 ms một lần, như đã đo).
- Không cần thư viện ngoài.

Nhược điểm

- Không tạo được một số lỗi: đĩa đầy, sập giữa lúc ghi. Lỗi quyền thì không ổn định (chạy bằng root hay trên Windows thì `chmod` không có tác dụng, như mình đã thấy).
- Vẫn là I/O thật: nếu thư mục tạm đầy hoặc bị một chương trình diệt virus khóa, test có thể đỏ vì lý do ngoài code.
- An toàn phụ thuộc vào kỷ luật: nếu lớp lưu trữ có đường dẫn mặc định, một test quên truyền đường dẫn vẫn chạm vào file thật.
- Phải tự lo dọn dẹp. Quên `t.after` thì rác tích dần trong `/tmp`.

B. File mẫu (fixture) lưu sẵn trong repo

Cách làm: để sẵn các file như `test/fixtures/bom.json`, `truncated.json`, `object-not-array.json`. Test đọc trực tiếp; test nào cần ghi thì copy sang thư mục tạm trước.

Ưu điểm

- Các ca hỏng được "trưng bày" rõ ràng, người mới mở thư mục ra là thấy ngay có những trường hợp nào.
- Dễ thêm ca mới: gặp file hỏng ngoài thực tế thì copy vào làm fixture.

Nhược điểm

- **Git và editor có thể âm thầm sửa fixture.** `core.autocrlf` đổi xuống dòng, editor xóa BOM hoặc tự thêm dòng trống cuối file. Khi đó test vẫn xanh nhưng không còn test đúng thứ bạn định test. Cần `.gitattributes` đánh dấu fixture là binary.
- Ký tự vô hình (BOM, zero-width) không nhìn thấy khi review code.
- Chỉ dùng được cho đọc. Ghi vẫn phải quay về phương án A.

C. File thật trên ổ RAM (tmpfs, `/dev/shm`)

Cách làm: giống A, nhưng thư mục tạm nằm trên ổ RAM.

Ưu điểm: giữ phần lớn độ trung thực của A, nhanh hơn một chút, không làm mòn ổ SSD.

Nhược điểm: chỉ có trên Linux. Một số hành vi khác ổ đĩa thường. Và với thời gian đã đo ở A, khoản tốc độ được thêm gần như không đáng kể ở quy mô dự án này.

D. Mock từng hàm của `fs` bằng `mock.method` (có sẵn trong `node:test`)

Cách làm: `mock.method(fs, 'readFileSync', () => '{}')` hoặc cho nó ném lỗi `ENOSPC`.

Ưu điểm

- Có sẵn, không cần cài thêm.
- Tạo được **mọi loại lỗi** một cách ổn định, kể cả những lỗi phương án A không làm được (đĩa đầy, `EACCES` khi chạy bằng root, ghi thất bại giữa chừng).
- Không chạm đĩa.

Nhược điểm

- **Phụ thuộc vào cách viết import**, như đã kiểm chứng: viết kiểu destructuring thì mock không có tác dụng và code đọc đĩa thật. Đây vừa là lỗ hổng về độ chính xác, vừa là lỗ hổng an toàn.
- Gắn chặt với cách cài đặt: đổi `readFileSync` thành `readFile`, hoặc thêm `existsSync` trước khi đọc, là test vỡ dù hành vi không đổi.
- Kiểm tra giả định của bạn về `fs`, không kiểm tra `fs` thật. Nếu bạn giả lập sai, chẳng hạn trả về chuỗi trong khi lẽ ra là Buffer vì quên `'utf8'`, test xanh nhưng code thật lại sai.
- Quên `mock.restoreAll()` thì mock rò sang các test khác trong cùng file.

E. Thay cả module `fs` bằng `mock.module`

Cách làm: chạy với cờ `--experimental-test-module-mocks`, thay `node:fs` bằng một object tự viết.

Ưu điểm: khắc phục nhược điểm của D vì bắt được cả kiểu destructuring. Không cần thư viện ngoài.

Nhược điểm: vẫn là tính năng **thử nghiệm** trên Node 22, API có thể đổi giữa các phiên bản. Mọi nhược điểm khác của D vẫn còn: gắn với cách cài đặt, chỉ kiểm tra giả định của bạn.

F. Thư viện file system trong bộ nhớ (`memfs`, `mock-fs`)

Cách làm: cài thư viện, tạo một cây thư mục ảo trong RAM, code đọc ghi lên đó.

Ưu điểm

- Có sẵn nhiều thao tác đúng kiểu file system (tạo thư mục, rename, liệt kê), không cần tự viết.
- Không chạm đĩa thật, nhanh.

Nhược điểm

- Thêm phụ thuộc, trái với mong muốn tránh framework phức tạp của bạn.
- Cũng chỉ là mô phỏng: mã lỗi, quyền truy cập, hành vi `rename` có thể khác bản thật ở những chỗ nhỏ, mà đó lại đúng là những chỗ bạn muốn test.
- Loại thư viện vá thẳng vào bên trong `fs` (như `mock-fs`) thường dễ hỏng khi nâng cấp Node.

G. Tiêm phụ thuộc (dependency injection): lớp lưu trữ nhận vào các hàm I/O

Cách làm: `createStorage({ readFile, writeFile, rename })`. Code thật truyền vào hàm của `fs`. Test truyền vào một bản giả viết tay: một `Map` trong bộ nhớ, hoặc một hàm luôn ném `ENOSPC`.

Ưu điểm

- Tạo được mọi lỗi giống D, nhưng **không phụ thuộc cách import** và không cần tính năng thử nghiệm.
- An toàn theo thiết kế: nếu bản giả không chạm đĩa thì test không thể chạm đĩa.
- Code nói rõ lớp lưu trữ phụ thuộc vào những gì.

Nhược điểm

- Thêm một tầng gián tiếp, code dài hơn một chút.
- Bản giả vẫn có thể khác `fs` thật. Bạn vẫn cần một số test chạy với `fs` thật (tức phương án A) để chắc phần "nối dây" đúng.
- Nếu tiêm quá chi tiết (từng hàm `fs` một), test cũng sẽ gắn với cách cài đặt như D.

H. Tách phần xử lý thuần túy khỏi phần I/O

Cách làm: chia lớp lưu trữ thành hai phần:

- `parseTickets(text)` và `serializeTickets(data)`: hàm thuần, nhận và trả về chuỗi.
- `load` / `save`: vài dòng gọi `fs` rồi chuyển chuỗi cho hai hàm trên.

Ưu điểm

- **Phần lớn các ca "file hỏng" thực ra là ca "chuỗi hỏng"**: BOM, JSON cắt ngang, `{}`, id trùng, `nextId` sai. Những ca này test được trên hàm thuần, không cần file, không cần mock, chạy cực nhanh và không có rủi ro an toàn nào.

- Không gắn với cách đọc file.

Nhược điểm

- Không tự trả lời được những câu hỏi thuộc về I/O: thư mục thay vì file, ghi nguyên tử, không được ghi đè khi lỗi. Vẫn phải kết hợp với một phương án khác cho phần mỏng còn lại.
- Đây là một quyết định thiết kế, không chỉ là cách test. Nếu code đã viết xong thì phải sửa lại cấu trúc.

I. Bản giả của cả lớp lưu trữ, kèm "contract test"

Cách làm: khi test lớp logic và CLI, thay toàn bộ lớp lưu trữ bằng `InMemoryStore`. Để chắc bản giả không lệch khỏi bản thật, viết **một bộ test hành vi chung** và chạy bộ đó cho cả `InMemoryStore` lẫn `JsonFileStore`.

Ưu điểm

- Các lớp phía trên được test nhanh và không phụ thuộc vào file.
- Contract test đảm bảo bản giả và bản thật hành xử giống nhau trên những gì đã khai báo.

Nhược điểm

- Bản thân phương án này **không test lớp lưu trữ**. Nó chỉ giúp lớp lưu trữ không làm chậm test của các lớp khác. `JsonFileStore` vẫn cần A hoặc G.
- Với một CLI có 4 lệnh, đây có thể là thiết kế quá tay.
- Contract test chỉ phủ những hành vi chung. Các ca chỉ có ở file (BOM, EISDIR) bản giả không có, nên không kiểm tra được.

Bảng tổng hợp

	Trung thực	An toàn	Tạo lỗi I/O	Bền khi refactor	Tốc độ	Phụ thuộc thêm
A. File tạm	Cao	Trung bình[1]	Hạn chế	Cao	Nhanh	Không
B. Fixture	Cao	Cao (chỉ đọc)	Không	Cao	Nhanh	Không
C. Ổ RAM	Cao	Trung bình[1]	Hạn chế	Cao	Nhanh	Không (chỉ Linux)
D. `mock.method`	Thấp	**Thấp**[2]	Đầy đủ	Thấp	Rất nhanh	Không
E. `mock.module`	Thấp	Cao	Đầy đủ	Thấp	Rất nhanh	Cờ thử nghiệm
F. memfs/mock-fs	Trung bình	Cao	Khá	Trung bình	Rất nhanh	Có
G. Tiêm phụ thuộc	Trung bình	Cao	Đầy đủ	Trung bình	Rất nhanh	Không
H. Tách hàm thuần	Cao[3]	Cao	Không	Cao	Rất nhanh	Không
I. Bản giả + contract	(cho lớp khác)	Cao	Không	Cao	Rất nhanh	Không

[1] An toàn nếu đường dẫn luôn được truyền vào và không có giá trị mặc định.
[2] Một lần import kiểu destructuring là test chạm đĩa thật.
[3] Trung thực với phần xử lý nội dung, nhưng không phủ phần I/O.

Chọn theo ưu tiên của bạn

Các phương án này **không loại trừ nhau**. Câu hỏi thực tế là kết hợp những gì:

- **Ưu tiên tối thiểu thay đổi thiết kế:** A cho hầu hết các ca, B cho các mẫu file hỏng. Chấp nhận không test được đĩa đầy hay sập giữa chừng, hoặc bỏ qua có điều kiện.
- **Ưu tiên tốc độ và an toàn tuyệt đối, chấp nhận sửa thiết kế:** H cho phần lớn ca hỏng, G cho các lỗi I/O khó tạo, cộng thêm vài test A để kiểm tra phần nối dây với `fs` thật.
- **Ưu tiên test được mọi loại lỗi I/O mà không sửa thiết kế:** D hoặc E. Khi đó bạn phải quy định chặt cách import `fs` trong code, và chấp nhận test dễ vỡ khi refactor.

Dữ kiện còn thiếu để chốt là: **bạn có sẵn sàng tách `parseTickets` ra thành hàm riêng không, và bạn có thật sự cần test kịch bản đĩa đầy hoặc sập giữa lúc ghi không?** Hai câu trả lời này quyết định phương án nào hợp với bạn hơn.

Trả lời hai câu của bạn: (1) Mình đang làm theo TDD từ đầu, chưa có code nào, nên sẵn

sàng tách parseTickets thành hàm riêng. (2) Mình không cần test kịch bản đĩa đầy ở giai đoạn này, nhưng muốn chắc rằng ghi thất bại giữa chừng thì không làm mất dữ liệu cũ. Ngữ cảnh: dự án nhỏ, một người làm, ưu tiên test dễ hiểu và không vỡ khi refactor, không muốn cài thêm thư viện. Với ngữ cảnh này, nên kết hợp những phương án nào? Vì sao?

Cân nhắc đánh đổi giữa hai cách mô phỏng khi tái cấu trúc code. ›

Mình kiểm tra nhanh một cách tạo lỗi ghi bằng file system thật trước khi đề xuất, vì nó quyết định có cần đến mock hay không.

Verify a real-fs way to make a write fail ›

Với ngữ cảnh của bạn, mình đề xuất kết hợp **H + A**, và xử lý kịch bản "ghi thất bại giữa chừng" bằng file system thật, **không cần đến mock hay tiêm phụ thuộc**. Kết quả vừa chạy cho thấy cách này khả thi: chỉ cần tạo sẵn một *thư mục* trùng tên với file tạm là lệnh ghi thất bại thật (`EISDIR`), kể cả khi chạy bằng root, và file gốc giữ nguyên.

Phương án được chọn

H — Tách `parseTickets` / `serializeTickets`: phần lớn test nằm ở đây

Gần như mọi ca "file hỏng" mà bạn đã liệt kê (BOM, JSON cắt ngang, `{}`, `null`, id trùng, id là chuỗi, `nextId` sai, `created_at` lỗi, trường lạ) thực chất là **ca "chuỗi hỏng"**. Test chúng trên hàm thuần mang lại đúng những gì bạn ưu tiên:

- **Dễ hiểu:** mỗi test chỉ có ba phần: một chuỗi đầu vào, một lần gọi hàm, một kết quả mong đợi. Không có thư mục tạm, không có bước dọn dẹp.
- **Không vỡ khi refactor:** test không biết file được đọc bằng cách nào.
- **Ký tự vô hình hiện rõ trong code:** viết `'\uFEFF[]'` ngay trong file test thì ai đọc cũng thấy. Đây là lý do mình **không chọn B (fixture)**: Git hoặc editor có thể âm thầm xóa BOM trong file fixture mà test vẫn xanh.

Vì bạn đang làm TDD từ đầu, việc tách hàm này gần như không tốn thêm chi phí. Ở dự án đã có sẵn code, đây mới là nhược điểm lớn của phương án H.

A — File thật trong thư mục tạm: chỉ cho phần I/O mỏng còn lại

Sau khi tách H, `load` và `save` chỉ còn vài dòng. Khi đó những gì cần test với file thật rất ít:

```
[ ] load: file chưa tồn tại              -> trả về dữ liệu rỗng
[ ] save rồi load                        -> nhận lại đúng dữ liệu, kể cả tiếng Việt
có dấu
[ ] load: đường dẫn là một thư mục       -> lỗi có code rõ ràng (EISDIR thật)
[ ] save thành công                      -> không còn sót file .tmp
[ ] save thất bại khi ghi file tạm       -> file gốc giữ nguyên từng byte
```

Ngoài ra còn một test ở tầng ghép lệnh (hoặc tầng CLI) cho quy tắc "load gặp file hỏng thì không được gọi save". Quy tắc này thuộc luồng xử lý lệnh, không thuộc lớp lưu trữ, nên đặt test ở đó.

Kịch bản ghi thất bại: tạo bằng tình huống thật

Để dữ liệu cũ không mất, `save` cần ghi kiểu nguyên tử: ghi toàn bộ nội dung mới ra `tickets.json.tmp`, rồi `rename` đè lên `tickets.json`. Lỗi xảy ra trước bước `rename`, dù ở thời điểm nào, đều không chạm tới file gốc.

Test:

```js
test('save: ghi file tạm thất bại thì file gốc giữ nguyên', (t) => {
  const file = tempFile(t, '[{"id":1}]');
  fs.mkdirSync(file + '.tmp');                    // chặn đường ghi file tạm
  const before = fs.readFileSync(file);

  assert.throws(() => save(file, []));
  assert.ok(before.equals(fs.readFileSync(file)));
});
```

Test này có một điểm hay: nếu `save` ghi thẳng vào file gốc (không nguyên tử), nó sẽ không bao giờ đụng tới đường `.tmp`, nên sẽ **không ném lỗi** và test **đỏ**. Như vậy test bắt được đúng lỗi thiết kế mà bạn đang lo.

Ba điểm cần lưu ý khi viết `save`

1. File tạm phải nằm cùng thư mục với file gốc. Đừng đặt nó trong `os.tmpdir()`. `rename` giữa hai ổ đĩa (hoặc hai phân vùng) sẽ thất bại với `EXDEV`, và trên một số hệ thống khi đó thao tác không còn nguyên tử.

2. Bước dọn dẹp không được che mất lỗi gốc. Kết quả vừa chạy cho thấy `fs.rmSync(tmp, {`

`force: true })` vẫn ném lỗi `ERR_FS_EISDIR` khi `tmp` là thư mục. Nếu `save` viết:

```js
} catch (err) {
  fs.rmSync(tmp, { force: true });  // ném lỗi mới ở đây
  throw err;                        // không bao giờ tới được dòng này
}
```

thì người dùng sẽ thấy lỗi của bước dọn dẹp, không phải lỗi thật. Hãy bọc bước dọn dẹp trong `try/catch` riêng rồi mới ném lại `err`. Test ở trên sẽ phát hiện được lỗi này nếu bạn kiểm tra thêm `{ code: 'EISDIR' }` trong `assert.throws`.

3. Đánh đổi chấp nhận được: test biết tên file tạm là `tickets.json.tmp`. Nếu sau này bạn đổi sang tên ngẫu nhiên, test sẽ đỏ, nhưng **đỏ rõ ràng chứ không âm thầm xanh**, nên không nguy hiểm. Với CLI chạy một người dùng, tên cố định là đủ, vì chuyện chạy song song đã được ghi nhận là nằm ngoài phạm vi.

Các phương án không chọn và lý do

Phương án	Lý do không chọn trong ngữ cảnh này
B. Fixture	H đã phủ các ca nội dung, lại tránh được rủi ro Git/editor sửa ngầm file
C. Ổ RAM	Phương án A đã đủ nhanh, C chỉ chạy được trên Linux
D. `mock.method`	Vỡ khi refactor, và một lần import kiểu destructuring là test chạm đĩa thật
E. `mock.module`	Vẫn là tính năng thử nghiệm, vẫn gắn với cách cài đặt
F. memfs	Phải cài thêm thư viện
G. Tiêm phụ thuộc	Thêm một tầng thiết kế vĩnh viễn chỉ để phục vụ một test, trong khi file thật đã tạo được lỗi đó
I. Bản giả + contract	Lớp logic vốn đã làm việc với mảng thuần nên test đã nhanh; với 4 lệnh thì làm thế này là quá tay

G là phương án dự phòng. Nếu sau này bạn cần giả lập lỗi mà file system thật không tạo được, chẳng hạn ghi được một nửa rồi mới lỗi, hay đĩa đầy, thì khi đó hãy cho `save` nhận hàm ghi qua tham số. Tới lúc cần hãy làm, chưa cần làm từ bây giờ.

Phân bổ số test dự kiến

```
parseTickets / serializeTickets   ~15 test   hàm thuần, không I/O
load / save                       ~5 test    file thật, thư mục tạm
luồng lệnh + CLI                  vài test   TICKETS_FILE trỏ tới file tạm
```

Theo TDD, bạn nên bắt đầu với `parseTickets('')`. Đây là test đơn giản nhất của lớp lưu trữ, và nó buộc bạn quyết định ngay hai điều đã treo từ trước: định dạng file là mảng trần hay `{ nextId, tickets }`, và file rỗng nghĩa là "chưa có dữ liệu" hay "file hỏng".

23 minutes ago

Tóm tắt giúp mình: vấn đề gốc, giải pháp đã chọn, lý do loại các phương án khác, rủi ro còn lại và cách giảm rủi ro.

Chuẩn bị bản tóm tắt ngắn gọn và đề xuất lưu lại.

Vấn đề gốc

Lớp lưu trữ của Ticket CLI cần được test sao cho đạt cả bốn yêu cầu: **chạy nhanh, không bao giờ chạm vào file dữ liệu thật, chứng minh được hành vi đúng khi file bị hỏng**, và **ghi thất bại giữa chừng thì không mất dữ liệu cũ**. Kèm theo đó là các ràng buộc: dự án nhỏ, một người làm, làm TDD từ đầu, ưu tiên test dễ hiểu và không vỡ khi refactor, không cài thêm thư viện.

Giải pháp đã chọn: H + A

H — Tách hàm thuần. `parseTickets(text)` và `serializeTickets(data)` nhận và trả về chuỗi. Phần lớn các ca "file hỏng" (BOM, JSON cắt ngang, không phải mảng, id trùng, `nextId` sai…) thực chất là ca "chuỗi hỏng", nên được test tại đây: khoảng 15 test, không có I/O.

A — File thật trong thư mục tạm cho phần `load` / `save` mỏng còn lại, khoảng 5 test. Mỗi test có thư mục riêng tạo bằng `mkdtempSync` và được dọn bằng `t.after`.

Ghi an toàn: `save` ghi ra `tickets.json.tmp` *cùng thư mục*, rồi `rename` đè lên file gốc. Kịch bản ghi thất bại được tạo bằng file system thật: tạo sẵn một thư mục trùng tên file tạm, lệnh ghi sẽ gặp `EISDIR` (đã kiểm chứng chạy được cả khi chạy bằng root). Nếu `save` ghi thẳng vào file gốc, test này sẽ đỏ.

An toàn cho dữ liệu thật: lớp lưu trữ không có đường dẫn mặc định. Chỉ CLI biết đường dẫn thật,

và có thể ghi đè bằng biến môi trường `TICKETS_FILE`.

Lý do loại các phương án khác

- **B. Fixture:** H đã phủ các ca nội dung. Ngoài ra Git hoặc editor có thể âm thầm xóa BOM hay đổi xuống dòng trong file fixture.
- **C. Ổ RAM:** không nhanh hơn đáng kể, lại chỉ chạy trên Linux.
- **D.** `mock.method`**:** vỡ khi refactor. Đã kiểm chứng: chỉ cần import kiểu destructuring là mock bị bỏ qua và code chạm đĩa thật.
- **E.** `mock.module`**:** vẫn là tính năng thử nghiệm trên Node 22, và vẫn gắn với cách cài đặt.
- **F. memfs/mock-fs:** phải cài thêm thư viện, và chỉ là mô phỏng.
- **G. Tiêm phụ thuộc:** thêm một tầng thiết kế vĩnh viễn chỉ để phục vụ một test mà file thật đã làm được. Giữ lại làm phương án dự phòng.
- **I. Bản giả + contract test:** quá tay cho 4 lệnh, vì lớp logic vốn đã làm việc với mảng thuần.

Rủi ro còn lại và cách giảm

Rủi ro	Cách giảm
Chỉ giả lập được lỗi ngay khi mở file tạm, chưa giả lập được "ghi được một nửa rồi lỗi"	Thiết kế ghi-file-tạm-rồi-rename đã bảo vệ file gốc dù lỗi xảy ra ở bất kỳ thời điểm nào trước `rename`. Khi thật sự cần test kịch bản này, chuyển sang G
Chương trình bị tắt đột ngột (kill, mất điện) chưa được test trực tiếp. Mất điện ngay sau `rename` có thể để lại file rỗng trên một số hệ thống	Chấp nhận ở giai đoạn này. Nếu cần chắc hơn, gọi `fsync` file tạm trước khi `rename`
Bước dọn dẹp che mất lỗi gốc (`rmSync` gặp thư mục sẽ ném `ERR_FS_EISDIR`)	Bọc bước dọn dẹp trong `try/catch` riêng. Test kiểm tra `{ code: 'EISDIR' }` để bắt được lỗi này
Test biết tên file tạm cố định	Nếu đổi tên, test sẽ đỏ rõ ràng chứ không âm thầm xanh. Chấp nhận
Lỡ tay chạm vào file thật	Lớp lưu trữ bắt buộc nhận đường dẫn. Test CLI luôn đặt `TICKETS_FILE`. Có thể thêm một test xác nhận gọi `load()` không có đường dẫn sẽ báo lỗi
Test file thật đỏ vì lý do môi trường (thư mục tạm đầy, bị khóa)	Giữ số test I/O ít. Mỗi test dùng thư mục riêng và luôn dọn dẹp
Lỗi quyền truy cập (`EACCES`) không test ổn định được khi chạy bằng root hoặc trên Windows	Bỏ qua có điều kiện và ghi rõ lý do, hoặc để dành cho G
Hai lệnh chạy cùng lúc ghi đè lên nhau (lost update)	Nằm ngoài phạm vi. Ghi rõ trong README là hạn chế có chủ đích
Khác biệt trên Windows (CRLF, `rename` đè file đang mở)	Ghi vào danh sách test, xử lý khi có người dùng Windows
Hai quyết định còn treo: định dạng file (mảng trần hay `{ nextId, tickets }`), và file rỗng nghĩa là "chưa có dữ liệu" hay "file hỏng"	Chốt ngay ở test đầu tiên: `parseTickets('')`

Nếu bạn muốn lưu bản tóm tắt này lại để tham khảo khi code, mình có thể chuyển nó thành một tài liệu.

Viết cho mình hàm save(filePath, tickets) bằng Node.js, ghi file JSON theo kiểu nguyên tử (ghi ra file tạm rồi rename), có xử lý lỗi.

Sắp xếp các tình huống kiểm thử trước khi viết mã. ›

Mình sẽ làm theo đúng quy trình đã bàn: viết test cho `save` trước, chạy để thấy đỏ, sau đó mới viết hàm.

Created 2 files, edited 2 files, and 4 more steps ›

Hàm `save` đã xong, kèm 9 test. Toàn bộ 10 test của dự án đều xanh. Mình viết test trước và chạy để thấy đỏ với stub rỗng, sau đó mới viết hàm.

Cách `save` hoạt động

1. Kiểm tra đầu vào: thiếu đường dẫn thì báo `INVALID_PATH`, dữ liệu không phải mảng thì báo `INVALID_DATA`. Ở bước này chưa có gì được ghi xuống đĩa.
2. Chuyển dữ liệu thành chuỗi JSON **trước khi đụng tới đĩa**.
3. Ghi ra `tickets.json.tmp` trong cùng thư mục, rồi gọi `fsync` để chắc dữ liệu đã thực sự

xuống đĩa.

4. `rename` đè file tạm lên file gốc.
5. Nếu bất kỳ bước nào lỗi: xóa file tạm (chỉ khi chính `save` đã tạo được nó), rồi ném lỗi `SAVE_FAILED`. Lỗi gốc được giữ trong `err.cause`, nên không bị bước dọn dẹp che mất.

Một lỗi mình mắc rồi tự sửa nhờ test

Phiên bản đầu tiên mở file tạm với flag `'wx'` (chỉ tạo mới, báo lỗi nếu đã tồn tại), với ý định không bao giờ ghi đè thứ không phải của mình. Nhưng nếu một lần chạy trước bị tắt đột ngột và để lại `tickets.json.tmp`, thì **mọi lần lưu sau đó sẽ thất bại mãi**. Mình viết test cho tình huống này trước, và nó đỏ đúng như dự đoán. Sau đó mình đổi sang flag `'w'`.

Mục tiêu an toàn ban đầu vẫn được giữ: biến `tmpCreated` chỉ bật sau khi mở file thành công. Vì vậy nếu ở đường dẫn tạm là một thứ không mở được, chẳng hạn một thư mục, `save` sẽ không bao giờ xóa nó. Test 4 kiểm tra đúng điều này.

Bộ test có thực sự bắt được lỗi không?

Mình cố tình làm hỏng code theo hai cách để kiểm tra:

Sửa sai	Test đỏ
Bỏ `try/catch` quanh bước dọn dẹp	Test 4 và 7: lỗi dọn dẹp che mất lỗi thật
Ghi thẳng vào file gốc, bỏ file tạm và `rename`	Test 4 và 5: file gốc bị phá khi ghi lỗi

Cả hai lỗi đều bị phát hiện, và đó là hai lỗi bạn lo nhất.

Giới hạn còn lại

- **Chưa `fsync` thư mục** sau `rename`. Trên Linux, nếu mất điện đúng lúc đó, thao tác `rename` có thể chưa được ghi nhận. Với CLI cá nhân thì chấp nhận được.
- **Quyền truy cập của file gốc không được giữ lại.** File mới mang quyền mặc định. Nếu sau này bạn đặt `chmod 600` cho dữ liệu, cần bổ sung phần này.
- **Trên Windows**, `rename` sẽ báo `EPERM` nếu file gốc đang được một chương trình khác mở (ví dụ editor). Khi đó `save` báo `SAVE_FAILED` và file gốc vẫn an toàn, nhưng chưa có test chạy trên Windows.
- Dòng `JSON.stringify(...)` là chỗ tạm. Khi viết `serializeTickets()` theo phương án H, hãy thay dòng đó bằng hàm mới. Các test của `save` sẽ không cần sửa.

Storage
JS
Download

Storage.test
JS
Download

Download all

20 minutes ago

Pasted text.txt
TXT

```
const fs = require('node:fs');
const path = require('node:path');                    // [SỬA 1] cần để lấy thư mục cha

function storageError(code, message, cause) {
 const err = new Error(message, cause ? { cause } : undefined);
 err.code = code;
 return err;
}

/**
 * Ghi danh sách ticket ra file JSON theo kiểu nguyên tử:
```

Show more

Weighing tmp-file strategies against symlink and crash-recovery risks. ›

Mình sẽ chạy lại phiên bản của bạn, rồi thử thêm vài tình huống mà cả 10 test chưa chạm tới.

Run user's version and probe symlink and typo-path cases ›

Bộ test báo `fail 1` có lẽ do cách mình truyền thư mục cho `node --test`, không phải do code của bạn. Mình chạy lại với đường dẫn file cụ thể.

Created a file, shared 2 files, ran 3 commands ›

Mình đã chạy lại phiên bản của bạn: 10/10 test xanh như bạn báo. Hai chỗ sửa của bạn đều đúng. Test 8 ("đường dẫn cha bị file chiếm chỗ") là một test hay, vì nó kiểm tra luôn việc `save` không phá file đang chặn đường.

Tuy vậy, mình tìm ra **hai lỗi an toàn thật và một vấn đề thiết kế**. Cả hai lỗi an toàn đều có từ **phiên bản của mình**, không phải do bạn sửa mà ra. Lỗi thứ hai thậm chí do chính mình tạo ra khi đổi flag `'wx'` thành `'w'`.

Lỗi 1 — `tickets.json` là symlink thì bị biến thành file thường

Đây là tình huống thực tế khá phổ biến: người dùng tạo symlink `~/.tickets.json` trỏ tới `~/Dropbox/tickets.json` để đồng bộ dữ liệu. `rename` thay **chính cái symlink** bằng một file thường. Kết quả chạy thử:

```
sau save, tickets.json còn là symlink? false | file thật: []
```

Dữ liệu mới nằm ở máy, bản trên Dropbox đứng yên. Không có lỗi nào được báo, và người dùng chỉ phát hiện khi mở máy khác lên.

Cách sửa: gọi `realpathSync` để lấy đường dẫn file thật, rồi đặt file tạm và `rename` ngay tại đó.

Lỗi 2 — File tạm là symlink thì `save` ghi đè lên file khác

Flag `'w'` đi theo symlink. Nếu `tickets.json.tmp` là symlink trỏ tới một file khác, file đó sẽ bị ghi đè:

```
quan-trong.txt sau save: "[\n  {\n    \"id\": 2\n  "
```

Với một CLI cá nhân, xác suất xảy ra thấp. Nhưng hậu quả là **phá dữ liệu nằm ngoài dự án**, nên vẫn đáng sửa.

Cách sửa: mở file với `O_NOFOLLOW`. Khi gặp symlink, lệnh mở báo `ELOOP` thay vì ghi xuyên qua. Vì `tmpCreated` vẫn là `false`, symlink đó cũng không bị xóa. Windows không có hằng số này nên mặc định dùng `0`.

Mình viết test cho cả hai lỗi trước và xác nhận chúng đỏ, rồi mới sửa. Có một chi tiết đáng kể: **lần sửa đầu tiên của mình làm test 8 của bạn đỏ.** `realpathSync` gặp `ENOTDIR` nhưng lại nằm ngoài `try`, nên lỗi lọt ra mà không được bọc thành `SAVE_FAILED`. Bản cuối đưa mọi bước chạm tới đĩa vào trong `try`. Đây là lần thứ hai trong cuộc trò chuyện này test bắt được lỗi trong chính bản sửa của mình.

Vấn đề thiết kế — Tự tạo thư mục sẽ che mất lỗi gõ nhầm đường dẫn

Đây không phải lỗi code, nhưng bạn nên cân nhắc:

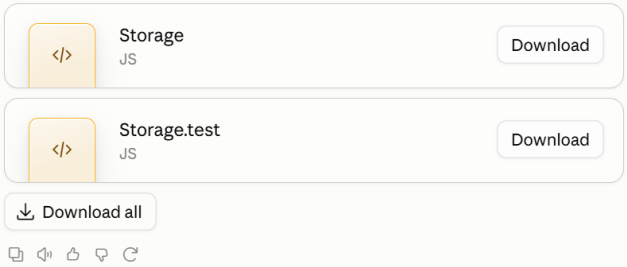
```
gõ nhầm -> tạo ra: [ 'hom' ] và lưu thành công, không báo gì
```

Nếu người dùng đặt `TICKETS_FILE=/hom/user/tikcets/tickets.json`, `save` sẽ âm thầm tạo cả cây thư mục sai rồi lưu vào đó. Lần chạy sau, nếu đường dẫn được sửa đúng, người dùng sẽ thấy danh sách trống và nghĩ rằng **dữ liệu đã mất**. Đây đúng là loại lỗi mà bạn đang cố tránh.

Một cách cân bằng là để **CLI** tự tạo thư mục **mặc định** (đường dẫn cố định, do chính bạn chọn), còn `save` thì không tạo gì. Như vậy lần chạy đầu trên máy mới vẫn chạy được, và đường dẫn do người dùng gõ sai sẽ được báo lỗi ngay. Mình giữ nguyên `mkdirSync` như bạn đã viết. Quyết định này là của bạn.

Những điều còn lại, ở mức chấp nhận được

- `tickets.json` **là symlink hỏng** (trỏ tới nơi không tồn tại): `realpathSync` báo `ENOENT`, nên `save` coi như file chưa có và thay symlink bằng file thường. Hiếm gặp, nhưng có thể thêm một test nếu bạn muốn xử lý chặt.
- **Thư mục được tạo không bị dọn đi nếu một bước sau đó thất bại.** Trong thực tế gần như không xảy ra, vì một thư mục vừa tạo thì hiếm khi có gì chặn được việc ghi file vào đó.
- Các giới hạn đã nêu trước vẫn giữ nguyên: chưa `fsync` thư mục, không giữ lại quyền truy cập của file gốc, Windows báo `EPERM` khi file đang được mở.



Reply

Claude is AI and can make mistakes.

Opus 5.5 Medium Manual